DSA0101 – Object oriented programming using C++

ASSIGNMENT 3 – CO3

Title: Implementation of Object-Oriented Programming Concepts and Operator Overloading in C++

Name: Sahhanaa Shree Thiyagarajan
Register Number: 192511283A
Programming Language: C++
Standard: C++17
Compiler: GCC (g++)
IDE: Visual Studio Code

---

1. Introduction

The objective of this assignment is to implement and analyze important C++ programming concepts, including constructors, dynamic memory allocation, destructors, deep copying, operator overloading, object comparison, and resource management.

The assignment consists of seven independent programs. Each program demonstrates a particular object-oriented programming concept through a practical example.

The programs are organized into separate folders, with each folder containing the corresponding C++ source file and a README file explaining compilation and execution.

Objectives

- Understand default, parameterized, overloaded, and delegating constructors.
- Implement dynamic memory allocation and deallocation.
- Understand shallow copying and deep copying.
- Implement copy constructors, assignment operators, and destructors.
- Overload arithmetic, comparison, increment, and stream operators.
- Apply object-oriented programming concepts to practical problems.
- Analyze memory usage, time complexity, and program behavior.
- Compile and test programs using a C++17-compatible compiler.

2. Software Requirements

Component	Specification
Programming language	C++
Language standard	C++17
Compiler	GCC (g++)
Development environment	Visual Studio Code
Operating system	Windows
Terminal	PowerShell

3. Project Structure

The assignment is divided into seven independent tasks.

```
None
DSA01-192511283A-Assignment3/
│
├── README.md
│
├── Task01/
│   ├── q1.cpp
│   └── README.md
│
├── Task02/
│   ├── q2.cpp
│   └── README.md
│
├── Task03/
│   ├── q3.cpp
│   └── README.md
│
├── Task04/
│   ├── q4.cpp
│   └── README.md
│
├── Task05/
```

```
│   ├── q5.cpp
│   └── README.md
│
├── Task06/
│   ├── q6.cpp
│   └── README.md
│
└── Task07/
    ├── q7.cpp
    └── README.md
```

4. Task 1 – Book Class and Constructors

4.1 Aim

To implement a Book class demonstrating different types of constructors, constructor delegation, initializer lists, and input validation.

4.2 Concepts Used

- Default constructor
- Parameterized constructor
- Constructor overloading
- Delegating constructor
- Member initializer lists
- Validation of input values

4.3 Design and Implementation

The Book class represents a book using attributes such as its title, author, and number of copies.

Different constructors initialize objects in different ways. A default constructor provides default values, while a parameterized constructor accepts values from the user or calling function.

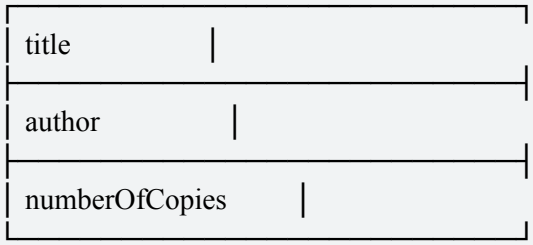
Constructor delegation allows one constructor to call another constructor of the same class. Initializer lists initialize data members directly before the constructor body executes.

Input validation prevents invalid values, such as a negative number of copies, from being accepted.

4.4 Memory Representation

None

Book Object



Each Book object maintains its own data members.

4.5 Complexity Analysis

- Object initialization: O(1), excluding the cost of copying strings.
- Accessing a data member: O(1).
- Memory usage: O(L), where L represents the total storage required for the strings in the object.

4.6 Expected Result

The program should demonstrate the creation of book objects using different constructors and show that valid values are accepted while invalid copy counts are rejected.

Conclusion: This task demonstrates how constructors simplify object initialization and how validation helps maintain valid object states.

5. Task 2 – Prescription Class and Deep Copying

5.1 Aim

To implement a Prescription class using a dynamically allocated array of medicine names and demonstrate deep copying, copy construction, copy assignment, and destruction.

5.2 Concepts Used

- Dynamic memory allocation
- Copy constructor

- Copy assignment operator
- Destructor
- Deep copying
- Self-assignment handling

5.3 Design and Implementation

The Prescription class stores medicine names in dynamically allocated memory.

A copy constructor creates a new object and allocates separate memory for its medicine list. The copy assignment operator releases the destination object's previous resources before copying the source data into newly allocated memory.

The destructor releases dynamically allocated memory when an object is destroyed.

Deep copying is important because two independent objects should not unintentionally share ownership of the same dynamically allocated array.

5.4 Memory Representation



Array A and Array B occupy separate memory locations.

5.5 Complexity Analysis

Let n be the number of medicines.

- Copy constructor: O(n).
- Copy assignment: O(n).
- Destruction: O(1) for releasing the array, excluding any element-specific destruction costs.
- Storage: O(n), excluding the lengths of individual medicine names.

5.6 Expected Output

```
None
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

The output demonstrates that independently managed prescription objects can contain different medicine lists after copying and assignment.

Conclusion: This task demonstrates deep copying and responsible management of dynamically allocated memory.

6. Task 3 – Auditorium Seat Management

6.1 Aim

To implement an auditorium seat map using dynamic memory allocation and demonstrate seat booking, cancellation, and object lifetime.

6.2 Concepts Used

- Dynamic two-dimensional allocation
- Pointers
- Constructor and destructor
- Seat booking
- Seat cancellation
- Local, block-scope, and dynamically allocated objects

6.3 Design and Implementation

The auditorium is represented by a two-dimensional seat map. Each position indicates whether a seat is available or booked.

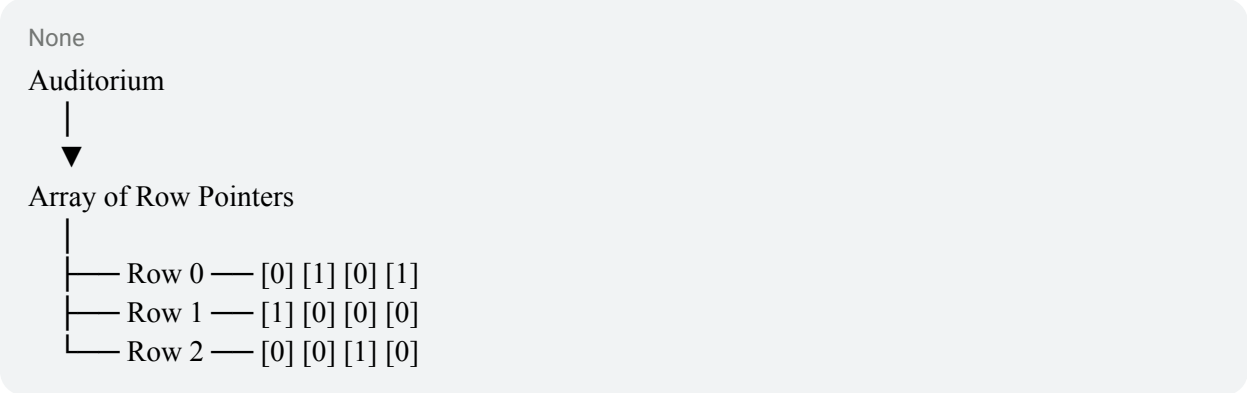
Dynamic allocation allows the number of rows and seats per row to be determined when an object is created.

The booking operation marks an available seat as occupied. The cancellation operation makes a previously booked seat available again.

The destructor releases the allocated memory after the auditorium object is no longer required.

Objects with automatic storage duration are destroyed when their scope ends. Dynamically allocated objects require explicit deletion when they are no longer needed.

6.4 Memory Representation



Here, 0 represents an available seat and 1 represents a booked seat.

6.5 Complexity Analysis

Let r be the number of rows and c be the number of seats per row.

- Creating the seat map: O(rc).
- Booking a seat: O(1).
- Cancelling a seat: O(1).
- Releasing the complete seat map: O(r + rc).
- Total storage: O(rc).

6.6 Expected Result

The program should display the seat map, demonstrate booking and cancellation, and show that allocated memory is released when the auditorium object is destroyed.

Conclusion: This task demonstrates dynamic two-dimensional memory management and the importance of matching allocation with deallocation.

7. Task 4 – Money Class and Operator Overloading

7.1 Aim

To implement a Money class that stores monetary values in paise and overloads arithmetic, comparison, and stream operators.

7.2 Concepts Used

- Operator overloading
- Addition and subtraction
- Compound assignment
- Equality and less-than comparison
- Input and output stream operators
- Integer-based monetary representation

7.3 Design and Implementation

The Money class stores the amount as an integer number of paise instead of using floating-point values as the primary representation.

For example, ₹200.00 is represented as 20,000 paise.

The overloaded addition and subtraction operators perform arithmetic on monetary values. The += operator updates an existing object. The comparison operators determine whether two amounts are equal or whether one is less than another.

The stream insertion operator (<<) displays the amount in rupees and paise. The stream extraction operator (>>) reads monetary input.

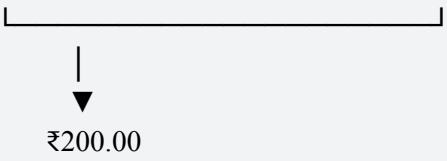
Representing money in the smallest currency unit helps avoid common floating-point precision problems.

7.4 Memory Representation

None

Money Object

```
┌──────────────────────┐
│ paise = 20000        │
```



7.5 Complexity Analysis

- Addition: O(1).
- Subtraction: O(1).
- Comparison: O(1).
- Compound assignment: O(1).
- Storage: O(1).

7.6 Expected Output

```
None
Rs 200.00
true
```

The exact formatting depends on the stream formatting implemented in the source code.

Conclusion: This task demonstrates how operator overloading makes user-defined monetary objects easier to manipulate and display.

8. Task 5 – Time Class and Playlist Management

8.1 Aim

To implement a Time class with prefix and postfix increment operators, comparison operators, and a Playlist class that supports checked element access.

8.2 Concepts Used

- Prefix increment (++object)
- Postfix increment (object++)
- Operator overloading
- Comparison operators

- Constant member functions
- Const and non-const access
- Bounds checking

8.3 Design and Implementation

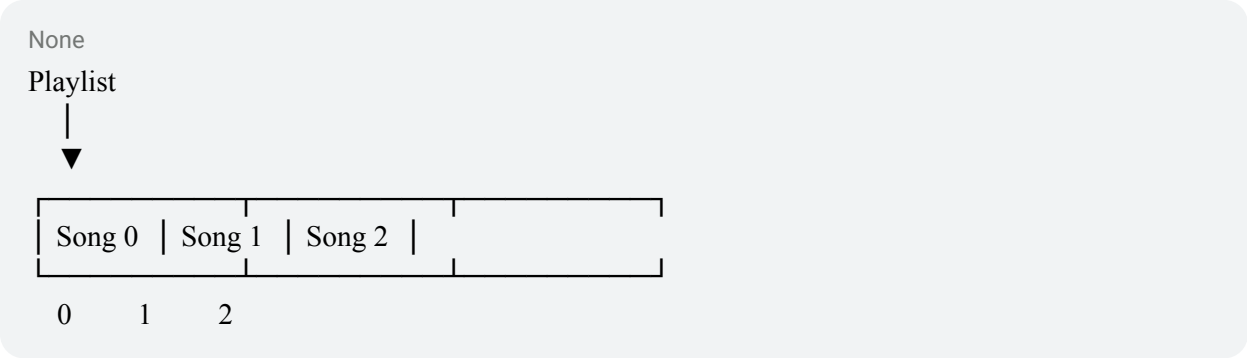
The Time class represents a time value and supports incrementing it using overloaded operators.

Prefix increment updates the object and returns the updated object. Postfix increment preserves the previous value for the expression and then increments the object.

Comparison operators allow two time objects to be compared.

The Playlist class stores a collection of entries. Its checked subscript operator provides access to an entry and validates the index before returning it. Separate const and non-const overloads allow read-only access to constant objects while permitting modification through non-constant objects.

8.4 Memory Representation



An invalid index should be detected rather than used to access memory outside the collection.

8.5 Complexity Analysis

- Prefix increment: O(1).
- Postfix increment: O(1) for the operation, excluding the cost of copying any larger user-defined state.
- Time comparison: O(1).
- Playlist element access: O(1), including a constant-time bounds check.
- Playlist storage: O(n), where n is the number of entries.

8.6 Expected Result

The program should demonstrate the difference between prefix and postfix increment, compare time values, and access playlist entries using valid indices.

Invalid indices should be handled safely according to the implementation.

Conclusion: This task demonstrates operator overloading and safer access to elements in a collection.

9. Task 6 – Text Class and String Operations

9.1 Aim

To implement a Text class using a dynamically allocated character array and overload operators for concatenation, equality, inequality, and output.

9.2 Concepts Used

- Dynamic character arrays
- Copy constructor
- Copy assignment operator
- Destructor
- Deep copying
- String concatenation
- Equality and inequality operators
- Stream insertion operator

9.3 Design and Implementation

The Text class manages its own dynamically allocated character array.

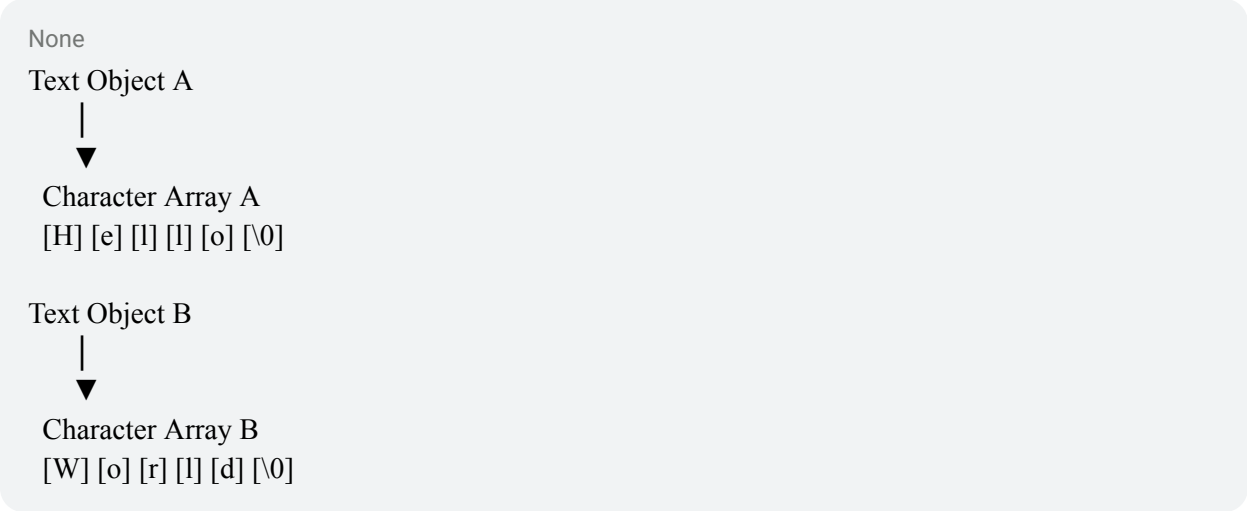
The constructor creates storage for the text. The copy constructor creates an independent copy of the original text. The copy assignment operator replaces an object's existing text with a copy of another object's contents.

The concatenation operator combines two text objects into a new text object. The equality and inequality operators compare their contents.

The destructor releases the allocated character array.

These operations require careful memory management to avoid memory leaks, double deletion, and accidental sharing of owned memory.

9.4 Memory Representation



Each object owns its character array independently.

9.5 Complexity Analysis

Let n and m represent the lengths of two strings.

- Copy constructor: O(n).
- Copy assignment: O(n), excluding any additional allocation costs.
- Concatenation: O(n + m).
- Equality comparison: O(min(n, m)) in the worst case up to the first differing character, with an additional length comparison as needed.
- Storage: O(n) for a string of length n.

9.6 Expected Result

The program should demonstrate text copying, concatenation, equality comparison, inequality comparison, and formatted output.

Conclusion: This task demonstrates dynamic character-array management and operator overloading for string-like objects.

10. Task 7 – Complex Number and Impedance Calculator

10.1 Aim

To implement a Complex class that overloads arithmetic, comparison, and output operators and applies complex-number operations to impedance calculations.

10.2 Concepts Used

- Complex-number representation
- Operator overloading
- Addition and subtraction
- Multiplication and division
- Equality comparison
- Stream output
- Magnitude calculation
- Series and parallel impedance
- Division-by-zero handling

10.3 Design and Implementation

A complex number is represented by its real and imaginary components:

[
z=a+bi
]

where (a) is the real component and (b) is the imaginary component.

The Complex class overloads arithmetic operators to perform operations on complex numbers. It also provides equality comparison and output formatting.

The magnitude of a complex number is calculated using:

[
|z|=\sqrt{a^2+b^2}
]

In an electrical circuit, series impedances are added:

[
Z_{\text{series}}=Z_1+Z_2
]

For two impedances connected in parallel:

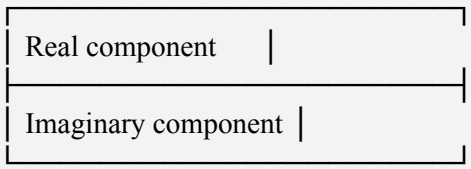
[
Z_{\text{parallel}}=\frac{Z_1Z_2}{Z_1+Z_2}
]

Division requires special handling when the divisor is zero. The implementation should detect this condition and report an error instead of performing invalid division.

10.4 Memory Representation

None

Complex Object



10.5 Complexity Analysis

- Addition: O(1).
- Subtraction: O(1).
- Multiplication: O(1).
- Division: O(1), provided the divisor is non-zero.
- Magnitude calculation: O(1).
- Series impedance: O(1).
- Parallel impedance: O(1), provided the denominator is non-zero.
- Storage: O(1).

10.6 Expected Result

The program should calculate and display complex-number arithmetic, magnitude, series impedance, and parallel impedance. It should also handle division-by-zero cases safely.

Conclusion: This task demonstrates how overloaded operators can simplify mathematical computations and how complex numbers can be applied to electrical impedance calculations.

11. Testing and Validation

Each program should be compiled and tested independently.

The following checks should be performed:

Task	Test cases
Task 1	Default construction, parameterized construction, constructor delegation, invalid copy count
Task 2	Copy construction, assignment, self-assignment, independent medicine lists
Task 3	Seat booking, cancellation, invalid seat selection, object destruction
Task 4	Addition, subtraction, compound assignment, comparisons, input and output
Task 5	Prefix increment, postfix increment, comparisons, valid and invalid playlist indices
Task 6	Copying, assignment, concatenation, equality, inequality, empty strings
Task 7	Arithmetic operations, magnitude, series impedance, parallel impedance, zero divisor

11.1 Compilation Commands

The following commands compile each program using C++17 and enable common compiler warnings.

```
None
g++ -std=c++17 -Wall -Wextra q1.cpp -o q1.exe
g++ -std=c++17 -Wall -Wextra q2.cpp -o q2.exe
g++ -std=c++17 -Wall -Wextra q3.cpp -o q3.exe
g++ -std=c++17 -Wall -Wextra q4.cpp -o q4.exe
g++ -std=c++17 -Wall -Wextra q5.cpp -o q5.exe
g++ -std=c++17 -Wall -Wextra q6.cpp -o q6.exe
g++ -std=c++17 -Wall -Wextra q7.cpp -o q7.exe
```

Each command must be run from its corresponding task directory.

11.2 Memory and Error Checking

Programs that allocate dynamic memory should be checked for memory leaks, invalid memory access, double deletion, and incorrect copy behavior.

Where supported, AddressSanitizer can be enabled during compilation using:

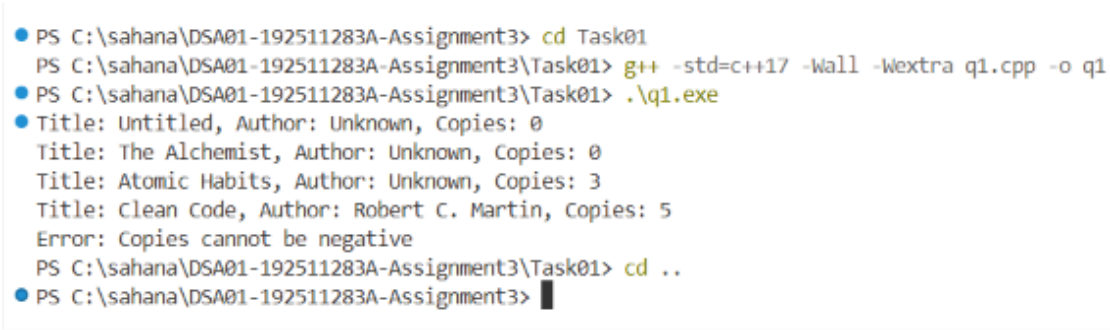
```
None
g++ -std=c++17 -Wall -Wextra -fsanitize=address -g q2.cpp -o q2
```

The same approach can be applied to the other relevant tasks.

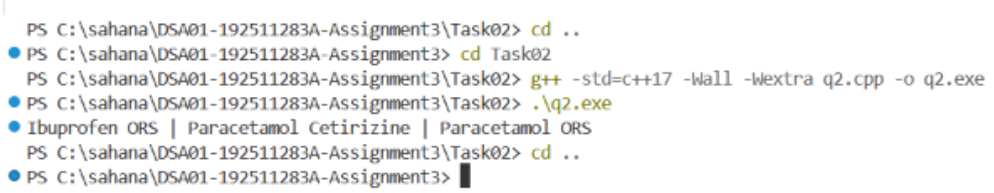
If AddressSanitizer is not supported by the installed compiler, a compatible GCC or Clang environment should be used. A normal compilation confirms that the program builds, but does not by itself prove that memory management is correct.

11.3 Output Screenshots

- **Task 1:** Screenshot of constructor demonstrations and validation.



- **Task 2:** Screenshot showing the expected medicine-list output.



- **Task 3:** Screenshot showing seat booking and cancellation.

```
PS C:\sahana\DSA01-192511283A-Assignment3> cd Task03
PS C:\sahana\DSA01-192511283A-Assignment3\Task03> g++ -std=c++17 -Wall -Wextra q3.cpp -o q3.exe
PS C:\sahana\DSA01-192511283A-Assignment3\Task03> .\q3.exe
Created event: College Fest (2 x 3)
Booked seat (0, 1)
Booked seat (1, 2)
Cancelled seat (0, 1)
Created event: Workshop (1 x 2)
Booked seat (0, 0)
Closing event: Workshop, booked seats remaining: 1
Created event: Concert (2 x 2)
Booked seat (1, 1)
Closing event: Concert, booked seats remaining: 1
Created event: Empty Event (0 x 5)
Closing event: Empty Event, booked seats remaining: 0
Closing event: College Fest, booked seats remaining: 1
PS C:\sahana\DSA01-192511283A-Assignment3\Task03> cd ..
```

- **Task 4:** Screenshot showing monetary operations

```
PS C:\sahana\DSA01-192511283A-Assignment3> cd Task04
PS C:\sahana\DSA01-192511283A-Assignment3\Task04> g++ -std=c++17 -Wall -Wextra q4.cpp -o q4.exe
PS C:\sahana\DSA01-192511283A-Assignment3\Task04> .\q4.exe
Rs 200.00
true
Rs 1.25
-Rs 1.00
PS C:\sahana\DSA01-192511283A-Assignment3\Task04> cd ..
PS C:\sahana\DSA01-192511283A-Assignment3>
```

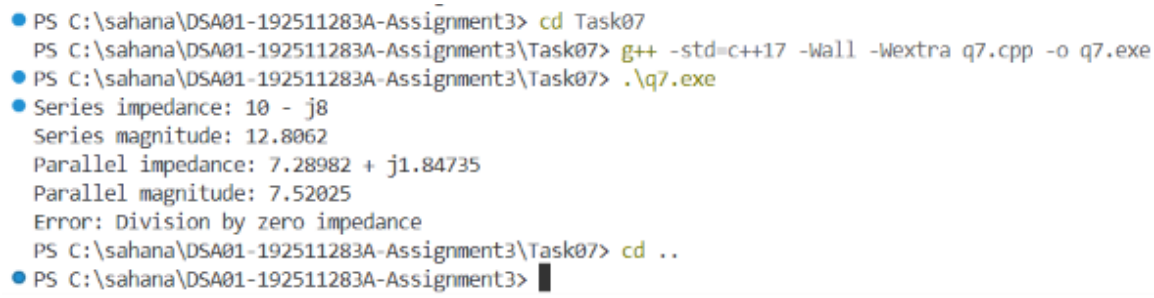
- **Task 5:** Screenshot showing increment operations and playlist access.

```
PS C:\sahana\DSA01-192511283A-Assignment3> cd Task05
PS C:\sahana\DSA01-192511283A-Assignment3\Task05> g++ -std=c++17 -Wall -Wextra q5.cpp -o q5.exe
PS C:\sahana\DSA01-192511283A-Assignment3\Task05> .\q5.exe
a = 0:59:58
b = 1:00:00
t = 1:00:00
a < b: true
Playing: Track One
Updated: Updated Track
Error: Playlist index out of range
PS C:\sahana\DSA01-192511283A-Assignment3\Task05> cd ..
PS C:\sahana\DSA01-192511283A-Assignment3>
```

- **Task 6:** Screenshot showing copying and text operations.

```
PS C:\sahana\DSA01-192511283A-Assignment3> cd Task06
PS C:\sahana\DSA01-192511283A-Assignment3\Task06> g++ -std=c++17 -Wall -Wextra q6.cpp -o q6.exe
PS C:\sahana\DSA01-192511283A-Assignment3\Task06> .\q6.exe
Hello, world!
Copy: Hello, world!
Equal: true
Empty text: []
Not equal: true
PS C:\sahana\DSA01-192511283A-Assignment3\Task06> cd ..
PS C:\sahana\DSA01-192511283A-Assignment3>
```

- **Task 7:** Screenshot showing complex arithmetic and impedance calculations.



12. Performance Analysis

The programs mainly perform operations on individual objects, arrays, or strings.

Most arithmetic and comparison operations in Tasks 1, 4, 5, and 7 have constant-time complexity. Tasks 2 and 6 require linear time for copying dynamically allocated collections or text. Task 3 requires time proportional to the number of seats when allocating or releasing the complete auditorium map.

Measured execution times should be recorded only after actually running the programs. For a meaningful comparison, compile all programs with the same optimization settings and execute them under similar conditions. For very small inputs, execution time may be dominated by program startup and measurement overhead.

Task	Main operation	Expected complexity
Task 1	Basic initialization	O(1), excluding string costs
Task 2	Deep copying n medicines	O(n)
Task 3	Initialize r × c seats	O(rc)
Task 4	Monetary arithmetic	O(1)
Task 5	Increment or indexed access	O(1)
Task 6	Concatenate strings of lengths n and m	O(n + m)
Task 7	Complex arithmetic	O(1)

13. Version Control and GitHub

The source files are maintained in a Git repository to track development and preserve the assignment structure.

The following commands can be used to commit and upload the files:

```
None
git add .
git commit -m "Add Assignment 3 solutions"
git push origin main
```

The repository includes the main README file, separate task folders, source code, and task-specific README files.

Progressive commits can be used to record the implementation and testing of individual tasks.

14. Results and Discussion

The assignment covers several important aspects of C++ object-oriented programming.

The first task demonstrates constructor-based initialization. The second and sixth tasks emphasize deep copying and resource ownership. The third task demonstrates dynamic allocation for a two-dimensional structure. The fourth and fifth tasks show how operator overloading can improve the usability of user-defined classes. The seventh task applies overloaded arithmetic operators to complex-number calculations and impedance formulas.

Together, the tasks demonstrate the importance of encapsulation, correct object initialization, safe memory management, and clear operator behavior.

The complexity analysis also shows that the cost of operations depends on the amount of data processed. Simple arithmetic is constant-time, whereas copying strings or initializing a complete seat map requires time proportional to the data size.

15. Conclusion

This assignment provided practical experience in implementing object-oriented programming concepts in C++.

Through seven independent programs, constructors, destructors, dynamic memory allocation, deep copying, operator overloading, and resource management were applied to practical examples. The programs also demonstrated the use of mathematical operations, comparison operators, input/output streams, and bounds checking.

The assignment strengthened the understanding of object lifetime, memory ownership, algorithmic complexity, and code organization. Separating each task into its own folder and documenting the compilation process also improved project maintainability and reproducibility.

16. References

1. Bjarne Stroustrup, *The C++ Programming Language*.
2. ISO C++ documentation and language references.
3. GCC documentation for compiler options and diagnostic warnings.
4. Assignment 3 – CO3 instructions provided for the course.